LAPORAN

PRAKTIKUM ALGORITMA DAN DATA STRUKTUR
(JOBSHEET 9)

POLITEKNIK NEGERI MALANG

OLEH

NAMA: ABRAR RAYHAN MUHAMMAD

NIM: 254107020217

NO ABSEN: 01

1. Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Membuat struktur data Stack
2. Menerapkan algoritma Stack ke dalam program Java

2. Praktikum

2.1 Percobaan 1: Mahasiswa Mengumpulkan Tugas

Waktu Percobaan : 90 Menit

Sejumlah mahasiswa mengumpulkan berkas tugas di meja dosen secara ditumpuk dengan menerapkan prinsip stack. Dosen melakukan penilaian secara terurut mulai dari berkas tugas teratas. Perhatikan Class Diagram Mahasiswa berikut.

Mahasiswa<NoAbsen>
nim: String nama: String kelas: String nilai: int
Mahasiswa<NoAbsen>() Mahasiswa<NoAbsen>(nim: String, nama: String, kelas: String) tugasDinilai(nilai: int)

Selanjutnya, untuk mengumpulkan berkas tugas, diperlukan class StackTugasMahasiswa yang berperan sebagai Stack tempat menyimpan data tugas mahasiswa. Atribut dan method yang terdapat di dalam class StackTugasMahasiswa merepresentasikan pengolahan data menggunakan struktur Stack. Perhatikan Class Diagram StackTugasMahasiswa berikut.

StackTugasMahasiswa<NoAbsen>
stack: Mahasiswa[] size: int top: int
StackTugasMahasiswa<NoAbsen>(size: int) isFull(): boolean isEmpty(): boolean push(mhs): void pop(): Mahasiswa peek(): Mahasiswa print(): void

2.1.1 Langkah-langkah Percobaan

A. Class Mahasiswa

1. Buat folder baru bernama Jobsheet9 di dalam repository Praktikum ASD. Buat file baru, beri nama Mahasiswa.java

2. Lengkapi class Mahasiswa dengan atribut yang telah digambarkan di dalam class diagram Mahasiswa, yang terdiri dari atribut nama, nim, kelas, dan nilai

3. Tambahkan konstruktor berparameter pada class Mahasiswa sesuai dengan class diagram Mahasiswa. Berikan nilai default nilai = -1 sebagai nilai awal ketika tugas belum dinilai

```java
Mahasiswa01.java
public class Mahasiswa01 {
    String nim, nama, kelas;
    int nilai;

    Mahasiswa01(String nama, String nim, String kelas){
        this.nama = nama;
        this.nim = nim;
        this.kelas = kelas;
        nilai = -1;
    }
}
Ctrl+L to chat, Ctrl+K to generate
```

4. Tambahkan method tugasDinilai() yang digunakan untuk mengeset nilai ketika dilakukan penilaian tugas mahasiswa

```java
void tugasDinilai(int nilai){
    this.nilai = nilai;
}
```

B. Class StackTugasMahasiswa

5. Setelah membuat class Mahasiswa, selanjutnya perlu dibuat class StackTugasMahasiswa.java sebagai tempat untuk mengelola tumpukan tugas. Class StackTugasMahasiswa merupakan penerapan dari struktur data Stack

6. Lengkapi class StackTugasMahasiswa dengan atribut yang telah digambarkan di dalam class diagram StackTugasMahasiswa, yang terdiri dari atribut stack, size, dan top

7. Tambahkan konstruktor berparameter pada class StackTugasMahasiswa untuk melakukan inisialisasi kapasitas maksimum data tugas mahasiswa yang dapat disimpan di dalam Stack, serta mengeset indeks awal dari pointer top

8. Selanjutnya, buat method isFull bertipe boolean untuk mengecek apakah tumpukan tugas mahasiswa sudah terisi penuh sesuai kapasitas

9. Pada class StackTugasMahasiswa, buat method isEmpty bertipe boolean untuk mengecek apakah tumpukan tugas masih kosong

10. Untuk dapat menambahkan berkas tugas ke dalam tumpukan Stack, maka buat method push. Method ini menerima parameter mhs yang berupa object dari class Mahasiswa

```java
public class StackTugasMahasiswa01 {
    Mahasiswa01[] stack;
    int top, size;

    public StackTugasMahasiswa01(int size){
        this.size = size;
        stack = new Mahasiswa01[size];
        top = -1;
    }

    public boolean isFull(){
        if (top == size -1) {
            return true;
        } else {
            return false;
        }
    }

    public boolean isEmpty(){
        if (top == -1) {
            return true;
        } else {
            return false;
        }
    }

    public void push(Mahasiswa01 mhs){
        if (!isFull()) {
            top++;
            stack[top] = mhs;
        } else {
            System.out.println("Stack penuh! Tidak bisa menambah tuas lagi.");
        }
    }
}
```

11. Penilaian tugas mahasiswa yang dilakukan oleh dosen dilakukan dengan menggunakan method pop untuk mengeluarkan tugas yang akan dinilai. Method ini tidak menerima parameter apapun namun mempunyai nilai kembalian berupa object dari class Mahasiswa

12. Buat method peek untuk dapat mengecek tumpukan tugas mahasiswa yang berada di posisi paling atas

13. Tambahkan method print untuk dapat menampilkan semua daftar tugas mahasiswa pada Stack

```java
    public Mahasiswa01 pop(){
        if (!isEmpty()) {
            Mahasiswa01 m = stack[top];
            top--;
            return m;
        } else {
            System.out.println(x: "Stack kosong! Tidak ada tugas untuk dinilai.");
            return null;
        }
    }

    public Mahasiswa01 peek(){
        if (!isEmpty()) {
            return stack[top];
        } else {
            System.out.println(x: "Stack kosong! Tidak ada tugas yang dikumpilkan.");
            return null;
        }
    }

    public void print(){
        for (int i = 0; i <= top; i++) {
            System.out.println(x: stack[i].nim + "\t" + stack[i].kelas);
        }
        System.out.println(x: "");
    }
```

C. Class Utama

14. Buat file baru, beri nama MahasiswaDemo.java

15. Tuliskan struktur dasar bahasa pemrograman Java yang terdiri dari fungsi main

16. Di dalam fungsi main, lakukan instansiasi object StackTugasMahasiswa bernama stack dengan nilai parameternya adalah 5.

17. Deklarasikan Scanner dengan nama variabel scan dan variabel pilih bertipe int

```java
import java.util.Scanner;

public class MahasiswaDemo01 {
    public static void main(String[] args) {
        StackTugasMahasiswa01 stack = new StackTugasMahasiswa01(size: 5);
        Scanner scan = new Scanner(System.in);    Resource leak: 'scan' is
        int pilih;    The value of the local variable pilih is not used


    }
}
```

18. Tambahkan menu untuk memfasilitasi pengguna dalam memilih operasi Stack dalam mengelola data tugas mahasiswa menggunakan struktur perulangan do-while

```
Mahasiswa01.java    StackTugasMahasiswa01.java    MahasiswaDemo01.java ×
MahasiswaDemo01.java > MahasiswaDemo01 > main(String[])
import java.util.Scanner;

public class MahasiswaDemo01 {
    Run | Debug
    public static void main(String[] args) {
        StackTugasMahasiswa01 stack = new StackTugasMahasiswa01(size: 5);
        Scanner scan = new Scanner(System.in);
        int pilih;

        do {
            System.out.println(x: "\nMenu");
            System.out.println(x: "1. Mengumpulkan Tugas");
            System.out.println(x: "2. Menilai Tugas");
            System.out.println(x: "3. Melihat Tugas Teratas");
            System.out.println(x: "4. Melihat Daftar Tugas");

            System.out.print(s: "Pilih");
            pilih = scan.nextInt();
            scan.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print(s: "Nama: ");
                    String nama = scan.nextLine();
                    System.out.print(s: "NIM: ");
                    String nim = scan.nextLine();
                    System.out.print(s: "Kelas: ");
                    String kelas = scan.nextLine();
                    Mahasiswa01 mhs = new Mahasiswa01(nama, nim, kelas);
                    stack.push(mhs);
                    System.out.printf(format: "Tugas %s berhasil dikumpulkan\n", mhs.nama);
                    break;
                case 2:
                    Mahasiswa01 dinilai = stack.pop();
                    if (dinilai != null) {
                        System.out.println("Menilai tugas dari" + dinilai.nama);
                        System.out.print(s: "Masukkan nilai (0-100): ");
                        int nilai = scan.nextInt();
                        dinilai.tugasDinilai(nilai);
                        System.out.printf(format: "Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
                    }
                    break;
                case 3:
                    Mahasiswa01 lihat = stack.peek();
                    if (lihat != null) {
                        System.out.println(x: "Tugas terakhir dikumpulkan oleh + lihat.nama");
                    }
                    break;
                case 4:
                    System.out.println(x: "Daftar semua tugas");
                    System.out.println(x: "Nama\tNIM\tKelas");
                    stack.print();
                    break;
                default:
                    System.out.println(x: "Pilihan tidak valid.");
            }
        } while (pilih >= 1 && pilih <= 4);

        scan.close();
    }
}
```

19. Commit dan push kode program ke Github

```
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git init
Initialized empty Git repository in D:/SEMESTER 2/Algoritma dan Struktur Data/Praktikum/Pertemuan 9/Tugas/.git/
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git add .
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git commit -m "Percobaan 1"
[master (root-commit) 3a1bbfa] Percobaan 1
 5 files changed, 137 insertions(+)
 create mode 100644 254107020217_ABRAR RAYHAN MUHAMMAD_1D TI_LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA.docx
 create mode 100644 Mahasiswa01.java
 create mode 100644 MahasiswaDemo01.java
 create mode 100644 StackTugasMahasiswa01.java
 create mode 100644 ~$4107020217_ABRAR RAYHAN MUHAMMAD_1D TI_LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA.docx
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git remote add origin https://github.com/abrar
rayhanmuhammad/algoritma-dan-struktur-data-pertemuan-9.git
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git branch -M main
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.26 MiB | 1.16 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/abrarrayhanmuhammad/algoritma-dan-struktur-data-pertemuan-9.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas>
```

20. Compile dan run program.

2.1.2 Verifikasi Hasil Percobaan

```
PS D:\SEMESTER 2\Algoritma dan Strukt
Struktur Data\Praktikum\Pertemuan 9\T
InExceptionMessages' '-cp' 'C:\Users\
887ba8b64cc\redhat.java\jdt_ws\Tugas_

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Dila
NIM: 1001
Kelas: 1A
Tugas Dila berhasil dikumpulkan

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Erik
NIM: 1002
Kelas: 1B
Tugas Erik berhasil dikumpulkan

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 3
Tugas terakhir dikumpulkan olehErik

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Tika
NIM: 1003
Kelas: 1C
Tugas Tika berhasil dikumpulkan
```

```
Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Dila    1001    1A
Erik    1002    1B
Tika    1003    1C


Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2
Menilai tugas dariTika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Dila    1001    1A
Erik    1002    1B
```

2.1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

3. Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

6. Commit dan push kode program ke Github

Jawaban:

1. Code yang perlu diperbaiki adalah method print pada file StackTugasMahasiswa01.java, perubahan pada kondisi perulangan fornya.

2. Data yang bisa ditampung berjumlah 5, dibawah ini potongan kodenya:

```
StackTugasMahasiswa01 stack = new StackTugasMahasiswa01(size: 5);
```

3. Agar dapat mencegah kelebihan data, dampak jika dihapus antara lain:

- Program error
- Bisa muncul ArrayIndexOutBoundException
- Data bisa berantakan karena melewati batas array

4. Penambawah method peekTerbawah() pada file StackaTugasMahasiswa01.java

```
    public Mahasiswa01 peekTerbawah(){
        if (!isEmpty()) {
            return stack[0];
        } else {
            System.out.println(x: "Stack kosong! Tidak ada tugas yang dikumpilkan.");
            return null;
        }
    }
```

Penambahan menu ke 5 dan penambahan case ke 5

```
System.out.println(x: "\nMenu");
System.out.println(x: "1. Mengumpulkan Tugas");
System.out.println(x: "2. Menilai Tugas");
System.out.println(x: "3. Melihat Tugas Teratas");
System.out.println(x: "4. Melihat Daftar Tugas");
System.out.println(x: "5. Melihat Tugas Terbawah");
```

```
case 5:
    Mahasiswa01 lihat2 = stack.peekTerbawah();
    if (lihat2 != null) {
        System.out.println("Tugas pertama dikumpulkan oleh " + lihat2.nama);
    }
    break;
```

5. Penambahan method count() pada file StackTugasMahasiswa01.java

```
public int count(){
    return top + 1;
}
```

Penambahan menu ke 6, penambah case ke 6 dan update kondisi while

```
System.out.println("\nMenu");
System.out.println("1. Mengumpulkan Tugas");
System.out.println("2. Menilai Tugas");
System.out.println("3. Melihat Tugas Teratas");
System.out.println("4. Melihat Daftar Tugas");
System.out.println("5. Melihat Tugas Terbawah");
System.out.println("6. Jumlah Tugas");
```

```
        case 6:
            System.out.println("Jumlah tugas saat ini: " + stack.count());
            break;
```

```
} while (pilih >= 1 && pilih <= 6);
```

6. Commit dan push kode program ke Github

```
Problems   Output   Debug Console   Terminal   Ports

PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git add .
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git commit -m "Pertanyaan precobaan 1
"
[main 4c647fd] Pertanyaan precobaan 1
 2 files changed, 30 insertions(+), 4 deletions(-)
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git psuh origin main
git: 'psuh' is not a git command. See 'git --help'.

The most similar command is
        push
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 679 bytes | 679.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/abrarrayhanmuhammad/algoritma-dan-struktur-data-pertemuan-9.git
   3a1bbfa..4c647fd  main -> main
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas>
```

2.2 Percobaan 2: Konversi Nilai Tugas ke Biner

Waktu Percobaan: 60 Menit

Sampai tahap ini, proses pengelolaan data tugas mahasiswa menggunakan konsep Stack telah berhasil dibuat pada Percobaan 1. Selanjutnya, pada Percobaan 2 ini ditambahkan method baru yang berfungsi untuk mengonversi nilai tugas bertipe int ke dalam bentuk biner setelah tugas tersebut diberi nilai dan dikeluarkan dari Stack.

2.2.1 Langkah-langkah Percobaan

1. Buka kembali file StackTugasMahasiswa.java

2. Tambahkan method konversiDesimalKeBiner dengan menerima parameter kode bertipe int

3. Tambahkan empat method yaitu isEmpty, isFull, push, dan pull sebagai operasi utama Stack pada class StackKonversi

```
    public String konversiDesimalKeBiner(int nilai) {
        StackKonversi01 stack = new StackKonversi01();
        while (nilai > 0) {
            int sisa = nilai % 2;
            stack.push(sisa);
            nilai = nilai / 2;
        }
        String biner = new String();
        while (!stack.isEmpty()) {
            biner += stack.pop();
        }
        return biner;
    }
}
```

```java
Mahasiswa01.java    StackTugasMahasiswa01.java M    StackK
StackKonversi01.java > StackKonversi01 > pop()
public class StackKonversi01 {
    int[] tumpukanBiner;
    int top, size;

    public StackKonversi01() {
        this.size = 32;
        tumpukanBiner = new int[size];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }

    public void push(int data) {
        if (isFull()) {
            System.out.println(x: "Stack penuh");
        } else {
            top++;
            tumpukanBiner[top] = data;
        }
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println(x: "Stack kosong.");
            return -1;
        } else {
            int data = tumpukanBiner[top];
            top--;
            return data;
        }
    }
}
```

4. Agar nilai tugas mahasiswa dikonversi ke dalam bentuk biner setelah dilakukan penilaian, maka tambahkan baris kode program pada method pop di class MahasiswaDemo

```java
case 2:
    Mahasiswa01 dinilai = stack.pop();
    if (dinilai != null) {
        System.out.println("Menilai tugas dari " + dinilai.nama);
        System.out.print(s: "Masukkan nilai (0-100): ");
        int nilai = scan.nextInt();
        scan.nextLine();
        dinilai.tugasDinilai(nilai);
        System.out.printf(format: "Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
        String biner = stack.konversiDesimalKeBiner(nilai);
        System.out.println("Nilai biner Tugas: " + biner);
    }
    break;
```

5. Compile dan run program.

6. Commit dan push kode program ke Github

```
Problems   Output   Debug Console   Terminal   Ports

PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git add .
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git commit -m "Percobaan 2"
[main b1bfca7] Percobaan 2
 3 files changed, 56 insertions(+), 1 deletion(-)
 create mode 100644 StackKonversi01.java
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 20 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 987 bytes | 987.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/abrarrayhanmuhammad/algoritma-dan-struktur-data-pertemuan-9.git
   4c647fd..b1bfca7  main -> main
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas>
```

2.2.2 Verifikasi Hasil Percobaan

```
887ba8b64cc\redhat.java\jdt_ws\Tu

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
5. Melihat Tugas Terbawah
6. Jumlah Tugas
Pilih: 1
Nama: Tika
NIM: 1001
Kelas: 1A
Tugas Tika berhasil dikumpulkan

Menu
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
5. Melihat Tugas Terbawah
6. Jumlah Tugas
Pilih: 2
Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87
Nilai biner Tugas: 1010111
```

2.2.3 Pertanyaan

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (kode != 0), bagaimana hasilnya? Jelaskan alasannya!

Jawaban :

1. Alur kerja:

1. Membuat Stack
   Stack dipakai untuk menyimpan sisa hasil pembagian
2. Proses Pembagian, desimal ke Sisa
   nilai % 2, ambil sisa 0 atau 1
   push ke stack
   nilai / 2, lanjut ke pembagian berikutnya
3. Mengambil dari Stack Membalik urutan
   Data yang terakhir masuk akan keluar duluan
4. Mengembalikan hasil

2. Program akan error, karena tidak ada variabel yang bernama kode

2.4 Latihan Praktikum

Waktu : 90 Menit

Mahasiswa mengajukan surat izin (karena sakit atau keperluan lain) setiap kali tidak mengikuti perkuliahan. Surat terakhir yang masuk akan diproses atau divalidasi lebih dulu oleh admin Prodi. Perhatikan class diagram berikut.

Surat<NoAbsen>
idSurat: String namaMahasiswa: String kelas: String jenisIzin: char durasi: int
Surat<NoAbsen>() Surat<NoAbsen>(idSurat: String, namaMahasiswa: String, kelas: String, jenisIzin: char, durasi: int)

Atribut **jenisIzin** digunakan untuk menyimpan keterangan ijin mahasiswa (S: sakit atau I: izin keperluan lain) dan **durasi** untuk menyimpan lama waktu izin.

Berdasarkan class diagram tersebut, implementasikan class **Surat** dan tambahkan class **StackSurat** untuk mengelola data Surat. Pada class yang memuat method main, buat pilihan menu berikut:

1. **Terima Surat Izin** untuk memasukkan data surat
2. **Proses Surat Izin** untuk memproses atau memverifikasi surat
3. **Lihat Surat Izin Terakhir** untuk melihat surat teratas
4. **Cari Surat** untuk mencari ada atau tidaknya surat izin berdasarkan nama mahasiswa

Jawaban :

Class Surat01

```
Surat01.java U ×
Surat01.java > ...
public class Surat01 {
    String idSurat, namaMahasiswa, kelas;
    char jenisIzin;
    int durasi;

    Surat01() {}

    Surat01(String idSurat, String namaMahasiswa, String kelas, char jenisIzin, int durasi) {
        this.idSurat = idSurat;
        this.namaMahasiswa = namaMahasiswa;
        this.kelas = kelas;
        this.jenisIzin = jenisIzin;
        this.durasi = durasi;
    }

    void tampil() {
        System.out.println(idSurat + " | " + namaMahasiswa + " | " + kelas + " | " + jenisIzin + " | " + durasi + " hari");
    }
}
Ctrl+L to chat, Ctrl+K to generate
```

Class StackSurat01

```
Surat01.java U    StackSurat01.java U ×    MainSurat01.java U    StackKon
StackSurat01.java > StackSurat01 > StackSurat01(int)
public class StackSurat01 {
    Surat01[] surat;
    int top, size;

    public StackSurat01(int size) {
        this.size = size;
        surat = new Surat01[size];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }

    public void push(Surat01 data) {
        if (isFull()) {
            System.out.println("Stack penuh");
        } else {
            top++;
            surat[top] = data;
        }
    }

    public Surat01 pop() {
        if (isEmpty()) {
            System.out.println("Stack kosong");
            return null;
        } else {
            Surat01 data = surat[top];
            top--;
            return data;
        }
    }

    public Surat01 peek() {
        if (isEmpty()) {
            System.out.println("Stack kosong");
            return null;
        } else {
            return surat[top];
        }
    }

    public void cari(String nama) {
        boolean ketemu = false;

        for (int i = top; i >= 0; i--) {
            if (surat[i].namaMahasiswa.equalsIgnoreCase(nama)) {
                surat[i].tampil();
                ketemu = true;
            }
        }

        if (!ketemu) {
            System.out.println("Surat tidak ditemukan");
        }
    }
}
```

Class MainSurat

```
urat01.java U    J StackSurat01.java U    J MainSurat01.java U ×    J MahasiswaDemo01.java
MainSurat01.java > MainSurat01 > main(String[])
import java.util.Scanner;

public class MainSurat01 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StackSurat01 stack = new StackSurat01(size: 5);

        int pilih;
        do {
            System.out.println(x: "\nMENU");
            System.out.println(x: "1. Terima Surat Izin");
            System.out.println(x: "2. Proses Surat Izin");
            System.out.println(x: "3. Lihat Surat Terakhir");
            System.out.println(x: "4. Cari Surat");
            System.out.println(x: "5. Keluar");
            System.out.print(s: "Pilih: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print(s: "ID Surat: ");
                    String id = sc.nextLine();
                    System.out.print(s: "Nama: ");
                    String nama = sc.nextLine();
                    System.out.print(s: "Kelas: ");
                    String kelas = sc.nextLine();
                    System.out.print(s: "Jenis Izin (S/I): ");
                    char jenis = sc.next().charAt(index: 0);
                    System.out.print(s: "Durasi: ");
                    int durasi = sc.nextInt();

                    Surat01 s = new Surat01(id, nama, kelas, jenis, durasi);
                    stack.push(s);
                    System.out.println(x: "Surat berhasil ditambahkan");
                    break;

                case 2:
                    Surat01 proses = stack.pop();
                    if (proses != null) {
                        System.out.println(x: "Memproses surat:");
                        proses.tampil();
                    }
                    break;

                case 3:
                    Surat01 lihat = stack.peek();
                    if (lihat != null) {
                        System.out.println(x: "Surat terakhir:");
                        lihat.tampil();
                    }
                    break;

                case 4:
                    System.out.print(s: "Masukkan nama mahasiswa: ");
                    String cari = sc.nextLine();
                    stack.cari(cari);
                    break;

                case 5:
                    pilih = 6;
                    break;

                default:
                    System.out.println(x: "Pilihan tidak valid");
            }

        } while (pilih >= 1 && pilih <= 5);

        sc.close();
    }
}
```

Output program jika dijalankan

```
MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 1
ID Surat: 1001
Nama: Abrar
Kelas: 1D
Jenis Izin (S/I): I
Durasi: 2
Surat berhasil ditambahkan

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 1
ID Surat: 1002
Nama: Rayhan
Kelas: 1E
Jenis Izin (S/I): S
Durasi: 2
Surat berhasil ditambahkan

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 3
Surat terakhir:
1002 | Rayhan | 1E | S | 2 hari

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 4
Masukkan nama mahasiswa: Rayhan
1002 | Rayhan | 1E | S | 2 hari

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 4
Masukkan nama mahasiswa: Abrar
1001 | Abrar | 1D | I | 2 hari

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 2
Memproses surat:
1002 | Rayhan | 1E | S | 2 hari

MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
Pilih: 4
Masukkan nama mahasiswa: Rayhan
Surat tidak ditemukan
```

```
MENU
1. Terima Surat Izin
2. Proses Surat Izin
3. Lihat Surat Terakhir
4. Cari Surat
5. Keluar
Pilih: 5
PS D:\SEMESTER 2\Algoritma dan Struktur Data\Praktikum\Pertemuan 9\Tugas>
```